

MasterHub PC Agent architecture and build guide

Workflow technology stack and implementation manual

The agent in agent_H lets a MasterHub controller operate applications and media on a separate Windows computer. The controller selects the device and sends a structured command. This agent validates the message, executes an allowed action on that computer, and returns an acknowledgement or an error.

This guide is for a developer who wants to understand the existing implementation and build a compatible agent. It explains the boundaries between the controller, transport, agent runtime and desktop handlers, then gives the message contracts, complete action catalogue, setup procedure and implementation sequence.

What is being built

The deliverable is a Python process running in an interactive Windows desktop session. It connects through an MQTT broker or a USB serial connection. Its browser automation uses Chrome, keyboard and mouse automation, and a separate Chrome DevTools connection for YouTube.

The agent contains no language model, natural language parser, React interface, HTTP server or database. The wire domain named ai_ml controls media; its name does not mean this project runs an AI model. A dashboard or AI command interpretation belongs in the external MasterHub controller.

Reading order

1. Architecture and stack explain the main building blocks and their responsibilities.
2. Startup, execution and protocol explain how a command moves through the system.
3. Action reference and browser implementation explain what the agent can do.
4. Setup, reconstruction and testing explain how to build and deploy another copy.
5. Reliability notes and source index help maintain or extend the implementation.

Project identity

Source root: D:\GALATIX\agent_H\agent

Project name in source: MasterHub PC Agent

Documentation date: 28 September 2026

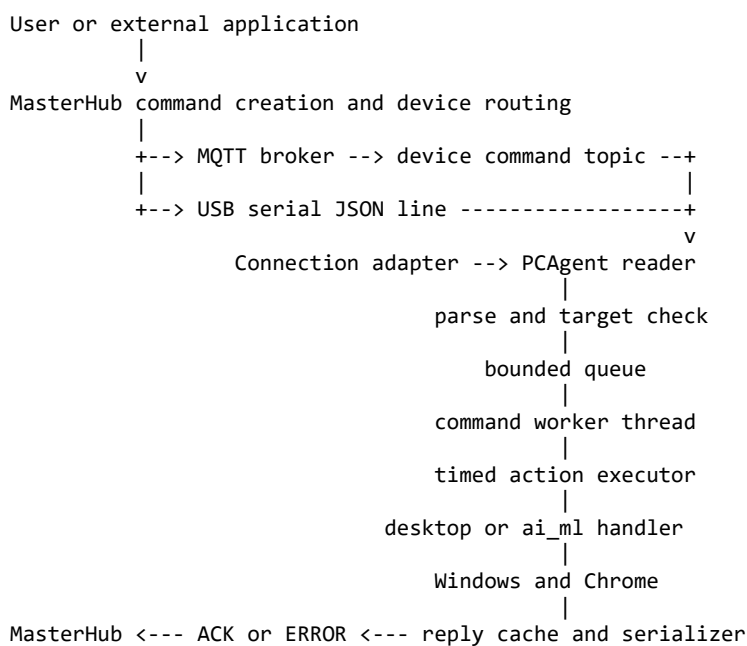
Source references throughout this guide are relative to the source root. Package requirements are the versions declared in requirements.txt, rather than a claim about the packages installed on any particular computer.

1 System architecture

Components and responsibility

Component	Responsibility
MasterHub controller	Select a target device, produce domain and action parameters, send commands, correlate replies and maintain device status. This component is external to the agent package.
MQTT broker or serial link	Carry protocol envelopes between controller and PC. The broker routes device topics; the serial transport carries newline delimited JSON.
PCAgent runtime	Announce identity, send heartbeats, parse input, filter targets, queue work, enforce capabilities and return ACK or ERROR.
Desktop and media handlers	Map a fixed action identifier to local Python functions. They call Chrome, Windows shortcuts, media keys or Chrome DevTools.
Windows desktop and Chrome	Provide the actual applications and user session on which the actions operate.

End to end flow



The transport is selected at startup; one PCAgent instance uses one connection. The abstraction is deliberately small: write(bytes), flush(), readline() and close(). pyserial.Serial, FakeSerial and MQTTConnection expose that same interface, so the execution logic does not need an MQTT-specific command path.

Sources: agent/pc_agent.py; mqtt/connection.py; mqtt/topics.py.

2 Technology stack and project structure

Actual stack used by this agent

Layer or dependency	Role and declared version
Python and standard library	Application runtime; threading, queue, concurrent.futures, json, dataclasses, subprocess, pathlib, logging and urllib. Installer documentation asks for Python 3.9 or newer; it does not enforce a minimum version.
paho-mqtt	MQTT client, authentication, TLS options, callbacks and network loop. Exactly 1.6.1 is pinned.
pyserial	USB serial reads and writes. Version 3.5 or newer.
pyautogui	Keyboard, mouse and media key automation. Version 0.9.54 or newer.
Pillow and pyscreeze	Image and screen support dependencies. Pillow 10.0.0 or newer; pyscreeze 0.1.29 or newer.
pygetwindow and pyperclip	Window control and declared clipboard support. Versions 0.0.9 and 1.8.2 or newer respectively; current handlers do not directly use pyperclip.
requests and urllib.request	Outbound HTTP, local DevTools discovery and optional device registration. requests 2.31.0 or newer.
websocket-client	Chrome DevTools Protocol requests over a WebSocket. Version 1.9.0 or newer.
python-dotenv	Read a local .env file without changing os.environ. Version 1.0.0 or newer.
pytest	Test framework used by tests/. It is not included in requirements.txt and must be installed separately for development.

What web stack means here

The web-facing pieces are outbound HTTP, JSON, MQTT and a local WebSocket connection to Chrome. There is no frontend build, npm package, REST service, ORM or agent-side database. The external controller's frontend framework, backend framework and database are separate architectural choices; this folder does not establish those choices.

Folder map

agent/pc_agent.py owns the runtime and CLI. protocol/ owns envelopes, validation and serialization. mqtt/ owns transport adaptation and topics. config/ owns configuration merging and validation. actions/desktop/ and actions/media/ own automation. common/ provides logging and exceptions. tests/ contains protocol, runtime, transport, configuration and action tests.

start_agent.bat and start_agent.py are the normal launcher. install_agent.bat creates the environment. register_and_start.py is an alternate registration launcher. Runtime data includes logs/pc_agent.log and actions/.runtime/youtube-chrome for the dedicated YouTube browser profile.

Sources: requirements.txt; imports in the implementation; start_agent.bat.

3 Configuration and startup workflow

Configuration precedence

Settings normally resolve from built-in defaults to `config/agent_config.json`, then the adjacent `agent-root .env` file, then the process environment, and finally explicit CLI overrides. The loader deep merges `transport.mqtt` and `automation`. Unreadable, missing or invalid JSON falls back to defaults, so a configuration typo can unexpectedly change startup behavior.

Within the merged environment mapping, `MASTERHUB_AGENT_DEVICE_ID` takes precedence over `DEVICE_ID`. Different aliases can therefore make an apparently conflicting device ID persist. A blank `MQTT_CLIENT_ID` selects `masterhub-agent-<device_id>`. Give each PC a unique identity and client ID.

Normal startup sequence

1. `start_agent.bat` changes to its own directory and checks for `.venv\Scripts\python.exe`. It asks the operator to run the installer if that environment is absent.
2. It checks imports for `dotenv` and `websocket`. If either is missing, it installs `requirements.txt`. It then launches `start_agent.py` with any supplied arguments.
3. `start_agent.py` loads and prints saved identity and transport settings. If its only argument is `--check-config`, it exits before importing the runtime. This is a settings preview, not full validation or a connection test.
4. The runtime parses CLI arguments, loads configuration, applies supported overrides and validates device ID, transport, capabilities and positive intervals and queue size. Startup validation does not check certificate or Chrome file existence.
5. If `register_url` is set, the runtime attempts HTTP registration. It then creates `MQTTConnection`, a real serial connection, or `FakeSerial`.
6. Capabilities `desktop` and `media` become wire domains `desktop` and `ai_ml`. `PCAgent` sends HELLO before starting heartbeat, worker and reader threads.
7. The main thread waits until shutdown. `Ctrl+C` sets the stop event, joins background threads briefly, requests executor shutdown and closes the connection.

Built in execution defaults

Setting	Fallback value and effect
<code>heartbeat_interval</code>	5 seconds between heartbeat attempts.
<code>command_timeout</code>	30 seconds waiting for an action future. This excludes time already spent in the command queue.
<code>command_queue_size</code>	100 waiting commands. The MQTT adapter has a separate unbounded inbox.
USB baud rate	115200; serial read timeout 1 second and write timeout 3 seconds.
MQTT runtime defaults	Port 1883, QoS 1, keepalive 60 seconds, publish timeout 5 seconds and connect wait 10 seconds. Set the broker explicitly.

The saved settings preview does not reflect later CLI overrides. Chrome also reads the default configuration at module import time, so `--config` alone does not reliably redirect the browser settings to a different file.

Sources: `config/loader.py`; `config/validator.py`; `start_agent.py`; `agent/pc_agent.py`.

4 Command execution and concurrency

The command lifecycle

1. The reader calls `connection.readline()`. For serial this reads a line; for MQTT it removes one payload from the adapter inbox.
2. `parse()` decodes JSON, validates required envelope fields and constructs an `Envelope`. Malformed protocol input is logged and discarded without a reply.
3. Non-command messages are ignored. A command addressed to another device is ignored without a reply. Valid wire commands require a nonempty target.
4. If `command_id` already has a cached outcome, the reader resends that outcome. Otherwise it tries to enqueue the envelope for up to 0.5 seconds. A full queue produces a cached `ERROR`.
5. The command worker removes one envelope and checks the reply cache again. This prevents already queued duplicates from running after an earlier copy completes.
6. The worker verifies that the domain is permitted and that a domain handler exists. It submits `handler.execute(action, params)` to a single-worker `ThreadPoolExecutor`.
7. It waits up to `command_timeout`. A dict with `success=true` becomes `ACK` with `status=success`. Exceptions, false success, invalid results, disallowed domains and timeouts become `ERROR`.
8. The reply is stored in a 256-entry in-memory `OrderedDict` and sent through the shared write lock. The oldest inserted entry is evicted when the limit is exceeded.

Threads and shared state

Execution context	Work performed
Main thread	Startup and shutdown supervision.
Reader thread	Receive, parse, target check, duplicate lookup and queue insertion.
Command worker thread	Sequentially consume the bounded queue and wait for action outcomes.
Executor thread	Run the potentially slow desktop or media action.
Heartbeat thread	Send heartbeat independently of action execution.
Paho network thread	Maintain MQTT connection and deliver callbacks when MQTT is selected.

`_write_lock` prevents heartbeat and command replies from interleaving on the connection. `_seen_lock` protects the reply cache. YouTube has its own `RLock` around browser operations.

Meaning of a successful reply

An `ACK` means the handler returned success. It does not universally prove the visible outcome. Several keyboard handlers return success after sending keys and waiting. The runtime does not include extra handler data such as pong, volume or results in the `ACK`.

A timeout replaces the executor, but cannot terminate the old Python thread. The old action can continue while a later command runs. Serial execution therefore holds during normal operation, but is not guaranteed after a timeout.

Source: `PCAgent` in `agent/pc_agent.py`.

5 Wire protocol and message examples

Envelope contract

All normal messages use protocol="masterhub", version="1.0", a message_type, a nonempty command_id and a timestamp in Unix seconds. The serializer omits optional fields whose value is None and omits empty params. For USB it encodes UTF-8 and appends a newline. MQTT preserves the serialized JSON payload; its message boundary supplies the framing.

Message type	Additional required fields checked by the validator
command	target, domain, action
ack	device_id, status
error	error
hello and heartbeat	device_id
status	device_id, status

The validator checks the protocol name and field presence, but does not enforce version, timestamp or full field types. A new controller should still produce the complete normal schema and a params object. Reusing a command ID intentionally asks the agent to replay the previous result while it remains cached.

Safe command and normal acknowledgement

```
{
  "protocol": "masterhub", "version": "1.0",
  "message_type": "command", "command_id": "demo_ping_001",
  "target": "PC_001", "domain": "desktop",
  "action": "ping_agent", "params": {},
  "timestamp": 1790553600
}

{
  "protocol": "masterhub", "version": "1.0",
  "message_type": "ack", "command_id": "demo_ping_001",
  "device_id": "PC_001", "status": "success",
  "timestamp": 1790553601
}
```

The timestamps above are illustrative. A real publisher should use its current time. On USB, write each object as one compact line followed by \n, rather than sending the formatted example across several lines.

Error and presence behavior

An error reply preserves the incoming command_id and normally includes device_id, status="error" and an error string. HELLO carries device_id and params.capabilities containing allowed wire domains. HEARTBEAT carries device_id; it does not set status="online". MasterHub must recognize presence from message type and update last-seen time.

COMMAND target validation happens before execution. Although the internal runtime check accepts None, the normal protocol validator rejects a missing or empty command target first. Untargeted broadcast commands are therefore not a supported wire feature.

Sources: protocol/envelope.py; messages.py; parser.py; serializer.py; validator.py.

6 MQTT and USB transport architecture

MQTT topic contract

Topic suffix under masterhub/devices/PC_001/	Direction	Behavior
command	Controller to agent	Agent subscribes only to its own exact topic; publish commands without retaining them.
ack	Agent to controller	Successful outcome; not retained.
error	Agent to controller	Failed outcome; not retained.
status	Agent to controller	HELLO, HEARTBEAT and STATUS share this retained topic. Each replaces the previous retained payload.

The adapter uses `clean_session=True`, default QoS 1, and reconnect delays from 1 to 30 seconds. On every successful connection it subscribes to the device command topic. Incoming topic checks reject anything outside that exact topic. Outgoing checks restrict publication to that device's status, ack and error topics.

The connection configures a retained Last Will before connect. The payload has `message_type="status"`, `command_id="lwt"`, `device_id`, `status="offline"` and `transport="mqtt"`. It is a manually built payload and has no timestamp. The broker can publish it after an unclean disconnect; controller heartbeat expiry must still handle stale devices.

The agent's clean close does not explicitly publish offline, and reconnect does not explicitly send a new HELLO. Heartbeats resume through the existing loop. A newly subscribing controller may therefore receive a retained heartbeat without the earlier HELLO capabilities.

Security boundary

TLS, CA files, client certificates and username/password authentication are supported. Client-side topic checks do not create broker ACLs. The broker administrator must restrict each credential to its assigned command subscription and result/status publications, and authorize the controller separately. TLS is opt-in in this implementation.

Serial behavior

`pyserial.Serial` opens the configured COM port at the configured baud rate with hardware and software flow control disabled. Use a real serial bridge and compatible wiring that present ports to the two endpoints. Connecting two ordinary USB host ports with a normal cable does not create the required serial link.

If USB is selected but no port is configured, the runtime silently chooses `FakeSerial` apart from a warning. `--fake-serial` applies only to the USB branch. `FakeSerial` captures writes and accepts injected reads; it is not a live link or a keyboard-driven command console.

Delivery boundaries

MQTT publication waits for completion where supported and records failure counts. On failure `write()` returns 0. `PCAgent` logs the short write but does not treat it as a failed reply or retry it. There is no durable outbox, persistent deduplication or guaranteed offline command storage. A controller retry must reuse the same command ID to benefit from the cache.

Sources: `mqtt/connection.py`; `mqtt/topics.py`; `agent/pc_agent.py`.

7 Complete desktop command reference

All actions on this page use domain="desktop". Empty parameters may be omitted or sent as {}. This catalogue is the dispatch table, including aliases, rather than every helper function in the source.

Action identifiers	Parameters	Behavior
ping_agent; get_agent_status	None	Safe handler checks. Runtime returns a normal success ACK, not the handler's detailed status dict.
open_notepad	None	Launch Notepad using Win+R.
write_notepad	text	Type into the focused window; empty text uses the built-in greeting.
save_notepad	filename	Ctrl+S and type a filename; default masterhub_note.txt. Depends on the current save dialog state.
open_calculator	None	Launch calc using Win+R.
open_calendar	None	Launch the outlookcal: URI using Win+R. Requires a registered application.
open_chrome	None	Launch configured executable and profile.
close_chrome	None	Send Alt+F4 to the active window. It does not identify or terminate a specific Chrome process.
search_chrome	query required	Open Chrome, focus its address bar and type the query.
previous_tab; next_tab	None	Ctrl+Shift+Tab or Ctrl+Tab in the focused application.
chrome_scroll_up; chrome_scroll_down	None	Find a Google Chrome window, restore and activate it, then scroll five units.
open_gmail; gmail_open	None	Open Gmail in the configured Chrome profile.
compose_gmail; gmail_compose	to, subject, body	Open a URL with prefilled compose fields. All three fields are optional. Does not send.
search_gmail; gmail_search	query	Open a Gmail search URL; empty query opens Gmail.
send_gmail; gmail_send	None	Ctrl+Enter in the focused window. Requires the correct draft and user session; success does not verify delivery.
open_youtube	None	Open or reuse a YouTube page in a dedicated DevTools-enabled Chrome profile.
search_youtube; play_youtube	query required	Search, click the first video result and attempt playback. Both actions start playback.
youtube_next; youtube_previous	None	Click the player next button, or invoke browser history.back().
youtube_volume_up; youtube_volume_down	None	Change the video element volume by five percentage points. The dispatch layer does not accept a custom step.
youtube_pause; youtube_mute	None	Toggle pause/play or toggle the video element's mute state.

open_outlook exists as a helper in system_apps.py but has no dispatch entry, so a remote open_outlook command returns an unknown-action error. Gmail helpers in chrome.py are also not the handlers used by the current dispatch table.

Source: actions/desktop/handler.py and its referenced modules.

8 Media commands and browser internals

Complete media action reference

These actions use wire domain="ai_ml". The configuration capability may be written as media or ai_ml; both enable this wire domain.

Action identifiers	Parameters	Behavior
open_jiosaavn	None	Open the JioSaavn website using Chrome.
search_jiosaavn	query required	Open a URL-encoded JioSaavn search path.
play; pause	None	Both send the same system playpause key; they toggle state.
next_track; previous_track	None	Send system nexttrack or prevtrack keys.
volume_up; volume_down	steps, default 2	Send that many system volume key presses. There is no explicit range validation.
mute	None	Toggle the system volume mute key.
album_search	query	Stub returning success and an empty result list. No music search API is implemented.

Two browser control approaches

General Chrome, Gmail and JioSaavn actions launch URLs or use GUI shortcuts. Chrome configuration is loaded into module-level constants. Gmail can search standard install locations if the configured executable is missing. Browser sign-in is supplied by the local Chrome profile; the agent does not implement Gmail OAuth or a Gmail API client.

YouTube uses Chrome DevTools Protocol, or CDP. youtube.py launches Chrome with a separate user-data directory, a dynamic debugging port and a loopback address of 127.0.0.1. Its exact profile directory is actions/.runtime/youtube-chrome, because the path is built from parents[2] of youtube.py.

The agent reads DevToolsActivePort, verifies /json/version, retrieves /json/list, and creates a YouTube page through PUT /json/new when needed. websocket-client connects to the selected tab's webSocketDebuggerUrl. CDP calls use Page.navigate, Page.bringToFront and Runtime.evaluate with JavaScript promises awaited.

YouTube search workflow

search_youtube URL-encodes the query, navigates to the results page, waits briefly, polls for the first video-result link and clicks it. It then polls for a ready video element on a /watch page and calls video.play(). Each polling phase can wait up to 15 seconds, so the combined operation can exceed the default 30-second command timeout.

Player controls target the DOM video element, avoiding keyboard focus for volume and playback. Selectors, consent pages and site changes can still break the flow. The dedicated YouTube profile does not share the ordinary Chrome profile's login automatically. stop_ad_monitor is a compatibility no-op, not an active ad-monitoring feature.

Sources: actions/media/handler.py; actions/desktop/youtube.py; gmail.py; chrome.py.

9 Install and configure another Windows PC

Prepare the target machine

Use an interactive Windows session with an unlocked desktop, Chrome installed and network access to the chosen broker. Install a Python version compatible with the declared dependencies. Copy the agent package; create a fresh virtual environment on each machine instead of copying another PC's .venv, credentials, logs or browser profile.

Run these PowerShell commands from the directory containing requirements.txt and install_agent.bat. Replace the example directory with the actual deployment directory.

```
Set-Location 'C:\MasterHubPCAgent'  
.\install_agent.bat
```

The installer checks Python on PATH, creates .venv, installs dependencies and tests imports of serial, pyautogui and requests. It does not configure MasterHub or the MQTT broker.

Configure MQTT in the agent root dotenv file

Create or edit .env beside start_agent.py. The following is a configuration template; replace the broker, device identity and credentials with your deployment values. Configure the broker for TLS before selecting port 8883.

```
DEVICE_ID=PC_001  
MASTERHUB_TRANSPORT=mqtt  
MQTT_BROKER=mqtt.example.internal  
MQTT_PORT=8883  
MQTT_CLIENT_ID=  
MQTT_USERNAME=pc001  
MQTT_PASSWORD=replace_with_device_secret  
MQTT_TLS_ENABLED=true  
MQTT_TLS_INSECURE=false  
MASTERHUB_AGENT_CAPABILITIES=desktop,media  
MASTERHUB_AGENT_HEARTBEAT_INTERVAL=5  
MASTERHUB_COMMAND_TIMEOUT=45  
MASTERHUB_COMMAND_QUEUE_SIZE=100
```

45 seconds is an example deployment adjustment for slower browser actions, not the current built-in default. Set MQTT_TLS_CA_CERTS when your broker requires a private CA. Add MQTT_TLS_CERTFILE and MQTT_TLS_KEYFILE only when client certificate authentication is configured.

Set local application configuration and start

Set automation.chrome_path, automation.chrome_profile and optional automation.chrome_extra_args in config/agent_config.json. Keep a unique DEVICE_ID on every computer. If you use a persistent MQTT client ID, change it when cloning the deployment.

```
.\start_agent.bat --check-config  
.\start_agent.bat
```

Check the displayed broker and identity, then verify a heartbeat and a ping_agent ACK from the controller. Establish the connection before trying visible desktop actions. The package has no installed Windows service, scheduled task, auto-update service or executable bundling pipeline.

Sources: install_agent.bat; start_agent.bat; SINGLE_START.txt; config/loader.py.

10 Configuration reference and controller integration

Main environment variables

Variables	Purpose
DEVICE_ID; MASTERHUB_AGENT_DEVICE_ID	Device identity; the prefixed name wins if both exist.
MASTERHUB_AGENT_DEVICE_NAME; MASTERHUB_AGENT_CAPABILITIES	Registration name and comma-separated desktop, media or ai_ml capabilities.
MASTERHUB_TRANSPORT; MASTERHUB_AGENT_PORT; MASTERHUB_AGENT_BAUDRATE	Select mqtt or usb, and configure the COM port and baud rate.
MASTERHUB_AGENT_HEARTBEAT_INTERVAL; MASTERHUB_COMMAND_TIMEOUT; MASTERHUB_COMMAND_QUEUE_SIZE	Heartbeat period, action wait timeout and bounded command queue size.
MQTT_BROKER; MQTT_PORT; MQTT_KEEPALIVE; MQTT_QOS; MQTT_CLIENT_ID	MQTT addressing and session settings.
MQTT_USERNAME; MQTT_PASSWORD	Broker credentials.
MQTT_TLS_ENABLED; MQTT_TLS_CA_CERTS; MQTT_TLS_CERTFILE; MQTT_TLS_KEYFILE; MQTT_TLS_INSECURE	Transport encryption and certificate settings.
MQTT_PUBLISH_TIMEOUT	Read from process environment by the connection builder or set through --mqtt-publish-timeout. This variable in .env alone is not mapped by config.loader.
MASTERHUB_AGENT_REGISTER_URL	Enable the runtime's optional /api/devices registration call.
MASTERHUB_AGENT_CHROME_PATH; MASTERHUB_AGENT_CHROME_PROFILE	Override local Chrome settings read during module import.

USB and diagnostic commands

```
.\.venv\Scripts\python.exe -m agent.pc_agent --help
.\start_agent.bat --transport usb --port COM4 --baudrate 115200
.\start_agent.bat --transport usb --fake-serial
```

Two registration paths

The runtime register_url option uses requests to POST /api/devices with device_id, name, device_type="computer", platform="windows", capabilities and transport="usb". That transport value is hardcoded even when the selected connection is MQTT. Network exceptions propagate, while HTTP errors are logged without raise_for_status().

register_and_start.py instead POSTs /api/v2/pc-agents/register with device_id, name, capabilities, transport="mqtt" and simulated=false, then starts an MQTT agent. It has deployment-specific defaults and explicit timeout/TLS choices, so supply and review its settings instead of treating it as identical to start_agent.bat.

The helper's failure message assumes MQTT HELLO can self-register a device, while protocol comments describe HELLO capabilities as a hint. Whether registration is automatic depends on the deployed MasterHub. Confirm the controller's API and presence behavior rather than relying on either comment as an integration guarantee.

Sources: config/loader.py; agent/pc_agent.py; register_and_start.py.

11 Build a compatible agent from scratch

Phase one implement the contract

Create the same package separation: protocol, config, transport, runtime and actions. Start with Envelope, MessageType, parse(), validate() and to_json_line(). Match existing field names and message values. Write round-trip tests before connecting any hardware. Add strict type checks as an intentional compatibility improvement.

Build FakeSerial with inject(), write(), readline(), flush() and close(). Create a PCAgent that can send HELLO, receive one ping_agent command and return an ACK with the same command_id. This provides a small working version before desktop automation introduces timing and focus problems.

Phase two add runtime controls

Add the reader, command queue and worker. Put all action execution behind execute(action, params). Add allowed-domain checks, unknown-action failures and the bounded reply cache. Add independent heartbeat scheduling and a shared connection write lock. Test duplicates, queue overload and failed handlers.

Add action timeouts with an explicit cancellation design. Matching the current executor replacement behavior preserves compatibility, but a process-based worker or cooperative cancellation is a better extension when actions must actually stop. Keep UI operations serialized across timeout recovery.

Phase three add real transports

Implement pyserial.Serial framing and verify HELLO and ping_agent across a real serial bridge. Then implement MQTTConnection against the same connection interface. Preserve exact topic names, QoS settings and ACK correlation. Add broker credentials, TLS, Last Will and reconnect subscriptions. Test with a private broker before connecting real desktop actions.

Phase four add automation

Start with calculator and Notepad. Add Chrome path/profile configuration and URL launch helpers. Implement Gmail compose and search through encoded URLs, with send treated as an explicit separate action. Add media key dispatch under ai_ml. Implement YouTube CDP discovery, WebSocket requests, selectors and playback waits.

Keep a clear contract for each function: validated inputs, explicit success or error, bounded waiting and a predictable window or browser target. Unit-test handler logic with mocks; then test visible behavior in a controlled Windows session.

Phase five integrate and package

Connect the controller's device registry and routing to this protocol. Build status tracking and command correlation on the controller side. Add startup configuration, dependency installation, logs and operator instructions. Preserve per-machine configuration when replacing code during updates.

Completion criteria

The rebuilt agent is ready for a controlled deployment when a fresh PC can install, announce its unique identity, receive only its own commands, run allowed actions, reject bad input, survive an action failure, return correlated outcomes and appear offline after connectivity is lost. Verify each criterion on both supported transports if both will be shipped.

Sources for implementation order: protocol/; agent/pc_agent.py; mqtt/; actions/; tests/.

12 Build the controller side and extend actions

Minimum controller behavior

The agent cannot choose a destination or interpret a sentence. A controller must turn a user request into target, domain, action and params. For example, a controller maps a request to play a song into target="PC_001", domain="desktop", action="play_youtube" and params.query containing the search text.

Maintain a device registry keyed by device_id, plus capabilities, transport and last-seen time. Track each outgoing command_id with target, submission time and expected result. Subscribe to status, ack and error topics before sending commands. Treat a device heartbeat timeout as offline and distinguish a transport timeout from an action failure.

Publish one command per JSON payload and do not retain command messages. Retained commands can unexpectedly execute after an agent reconnects. If you retry because a reply is missing, reuse the command ID, while recognizing that the agent cache is bounded and disappears on restart.

For a web dashboard, the browser should call an authenticated controller API. The controller performs authorization and routes commands to MQTT or serial. The agent package imposes no particular frontend or server framework. Any new UI, database or AI interpreter is an additional controller component, not an existing dependency of this agent.

Add a new action to an existing domain

1. Implement the Python function in the appropriate actions module. Return a dict with success=true only when the function's completion condition is satisfied; otherwise return success=false and error, or raise an exception.
2. Add a fixed dispatch entry to DesktopHandler or MediaHandler. Extract and validate supported params explicitly. Do not expose arbitrary Python or shell evaluation as a command.
3. Add controller-side mapping using exactly the same wire action identifier and domain. Define the user-facing parameter schema and any confirmation for consequential actions there.
4. Add tests for success, invalid input, handler failure and reply correlation. Then exercise the real UI manually in a test session.

Example extension pattern

```
def open_local_help(params):
    # Implement a specific, allowlisted operation here.
    return {"success": True, "action": "open_local_help"}

# Inside DesktopHandler.__init__, after its dispatch map exists:
self._dispatch["open_local_help"] = open_local_help
```

This snippet shows the registration pattern only; it does not implement opening a help document. Adding a new domain additionally requires updating _DOMAIN_HANDLERS, configuration capability validation, the capability-to-domain mapping, controller routing and tests.

Sources: actions/desktop/handler.py; actions/media/handler.py; agent/pc_agent.py; config/validator.py.

13 Testing and acceptance workflow

Existing automated test areas

Test file	What it covers
test_protocol.py	Envelope construction, round trips, required fields, malformed JSON, message types and newline framing.
test_pc_agent.py	HELLO, heartbeat, success and failure replies, targeting, capability restrictions, duplicates, queue order, overload and timeouts.
test_mqtt_connection.py	Mock MQTT connection, own-topic subscription, client identity, Last Will, routing, TLS options, topic checks, publish failure and metrics.
test_config_validator.py	Invalid identities, transports, baud rates, MQTT values, capabilities, timing values and certificate-file checks.
test_dotenv_config.py	Dotenv precedence, process overrides, no environment mutation and device-derived client identity.
test_command_updates.py	Mocked YouTube search/playback behavior, video volume control, Gmail profile selection and dispatch, and JioSaavn dispatch.

Run the suite in a development copy

```
Set-Location 'C:\MasterHubPCAgent'  
.\.venv\Scripts\python.exe -m pip install pytest  
.\.venv\Scripts\python.exe -m pytest tests\ -q
```

The tests use mocks and FakeSerial for most behavior. They do not establish real broker permissions, serial wiring, Gmail delivery, current YouTube compatibility or GUI focus correctness. Treat a passing suite as a software check and separately run the integration sequence below.

Integration acceptance sequence

1. Start the configured agent. Confirm a HELLO or heartbeat for the expected device. Verify MQTT broker logs show the correct client ID and topic subscription.
2. Send the ping_agent example and confirm ACK uses the same command_id. Send it again with the same ID and verify that the cached result is replayed.
3. Address a command to a different device and verify no action or reply from this agent. Try an unsupported action and verify ERROR.
4. Run open_calculator. Test Notepad open, write and save in order using a disposable output file. Check the actual window and file, not only the ACK.
5. Test Chrome and the dedicated YouTube profile. Verify search starts a video and volume targets that video. Test Gmail compose with a test draft; perform send acceptance only with a deliberate test recipient and operator approval.
6. Disconnect the broker or serial connection. Observe controller offline behavior, reconnect recovery and how missing replies are handled. Test restart and duplicate IDs to understand the cache boundary.
7. Add another PC with a different DEVICE_ID and verify the same command topic or serial route never operates both computers.

This is an acceptance procedure for the builder; automated and live integration results should be recorded when it is executed in the intended deployment.

14 Reliability limits and implementation improvements

Current behavior that affects correctness

Timeouts do not kill running actions. Old executor threads can continue, retain locks, change the UI later or delay process exit. Queue wait time is not part of the action timeout. Long YouTube waits can exceed the default limit. Prefer cancellation-aware operations or isolated worker processes with a deliberate UI recovery strategy.

Reply deduplication is temporary. It stores 256 outcomes in memory, does not persist across restart and does not prove exactly-once execution. A retry after cache eviction may repeat an operation. The cache also stores queue-full errors, so reusing that rejected ID returns the cached error while it remains present.

Transport errors are not fully supervised. The reader exits on a read exception without restarting itself or stopping the whole agent. An exception while sending a worker reply can end that worker. MQTT write failures return zero but the runtime still records the envelope as sent. Add explicit transport status propagation, retry/outbox policy and worker health supervision.

Input validation checks presence more than types. Some values can raise exceptions after `validate()`, during Envelope construction or later dictionary operations; those paths are not all caught as `ProtocolError`. Validate types, size limits, supported versions and action parameter bounds before queueing.

Resource and persistence limits

The bounded command queue does not bound MQTT's separate inbox. Runtime executed and sent_envelopes lists grow for the life of the process. `logs/pc_agent.log` has no rotation. Add caps, rotating logs and metrics before using long-running deployments with substantial command volume.

There is no durable job database, command expiry, cancellation protocol, signed envelope, local operator approval layer or per-user authorization inside this package. These are additional features to design if the deployment needs them. The timestamp field currently does not reject stale commands.

Desktop behavior that needs care

Many GUI actions assume the correct window has focus. `close_chrome` sends `Alt+F4`, `send_gmail` sends `Ctrl+Enter` and Notepad writes to the active window. Add application targeting and post-action verification for operations where a false success is unacceptable. Media play and pause are both toggles, as is `youtube_pause`.

The YouTube debugging endpoint is loopback-only, but its browser profile and DevTools port should remain local. Do not expose debugging to the network. Protect broker credentials and browser profiles, and configure broker-side ACLs rather than relying on topic checks in a client you do not control.

The runtime logs raw input and some handlers log search text or email metadata. Use redaction, retention limits and file access controls appropriate to the deployment. Configuration changes require a restart for module-level browser constants to refresh.

These improvements are separate from the implementation described in earlier sections. They should be tested and introduced intentionally rather than assumed to exist already.

15 Troubleshooting and source index

Troubleshooting map

Symptom	Checks and corrective direction
No device presence	Check actual identity, broker/port/TLS/authentication, topic permissions or serial port. Check for a FakeSerial warning.
Presence but no command reply	Check exact target and command topic, required protocol fields, queue and worker health. Review raw receive and parse logs.
Agent reports an action error	Check allowed domain, action spelling, required params and application availability. Preserve command_id when correlating logs.
ACK but wrong visible result	Check focus, dialog state, timing and toggle semantics. Keyboard dispatch alone does not verify the intended effect.
YouTube cannot find a tab or video	Open YouTube through the agent first. Check dedicated profile, DevTools endpoint, consent screens and DOM selectors.
Settings appear not to change	Check process environment and prefixed identity alias. Restart after edits. --check-config shows saved settings before CLI overrides.
Replies disappear during network loss	Inspect failed _publishes and broker delivery. There is no application-level reply retry or persistent outbox.
Commands fail after a disconnect	The serial reader may have exited; inspect thread health and restart the process after restoring the connection.

Source index for a developer

Source files	Use them to understand
agent/pc_agent.py	PCAgent, FakeSerial, threading, queue, reply cache, timeouts, CLI and connection construction.
protocol/envelope.py; messages.py; parser.py; validator.py; serializer.py	Protocol model, message builders, parsing, required fields and JSON framing.
mqtt/connection.py; mqtt/topics.py	Serial-like MQTT adapter, topics, TLS, Last Will, callbacks and telemetry.
config/loader.py; config/validator.py	Defaults, .env and environment precedence, JSON merging and validation.
actions/desktop/handler.py; actions/media/handler.py	Complete remote action dispatch and wire-domain mapping.
actions/desktop/chrome.py; gmail.py; notepad.py; system_apps.py; youtube.py	Concrete GUI shortcuts, URL launches, Chrome profile configuration and CDP implementation.
common/logger.py; common/exceptions.py	Logging destination and exception types.
start_agent.py; start_agent.bat; install_agent.bat; register_and_start.py	Startup, installation and the two registration paths.
requirements.txt; tests/; SINGLE_START.txt	Declared dependency versions, regression tests and deployment update notes.

The README is useful for the original USB design, but its coordinate-based YouTube description and early startup outline differ from the current implementation. Follow the source paths above for the behavior documented in this guide.